

自己学習ポートフォリオ

研修期間中に自主学習で制作したゲーム作品

トランプゲーム(CUI)

氏名：N.K.
株式会社アルテクナ
アウトソーシング事業部

概要

○開発内容

ブラックジャック、七並べ、
ババ抜き、UNO、大富豪

○開発目的

C言語で学んだ知識の定着

○開発期間

6日間

○開発人数

1名(個人開発)

○言語/UI

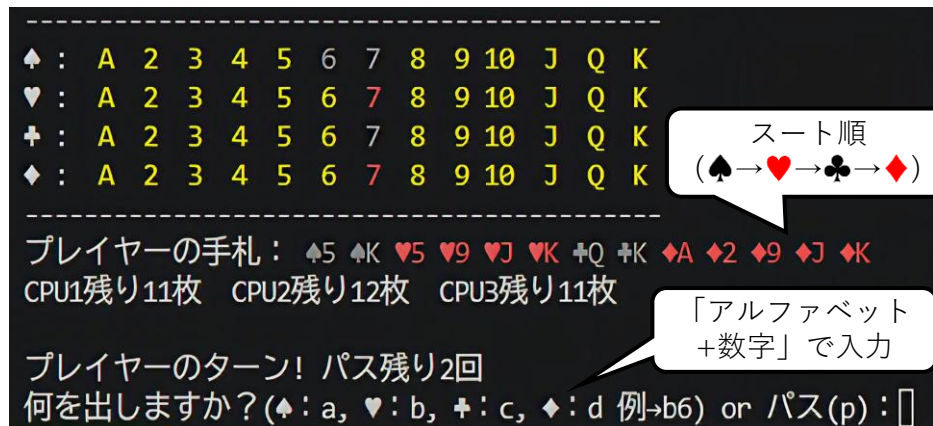
C言語/CUI

○ライブラリ

C標準ライブラリ

○開発環境

Visual Studio Code、
Linux(Ubuntu)



開発のポイント

○工夫した点

- ・手札をスート順(♠→♥→♣→♦)に自動整列する機能を実装
- ・「a1」のような入力によるカード選択機能を実装

○苦勞した点

- ・パスや上がりによるターン制御

問題点

- ・大富豪においてパスによる場の流しが反映されない。
- ・手札が0枚の人(上がり)のターンがスキップされない

解決策

パス：パスによるターン処理も別で行う
上がり：上がり判定を別で追加する

開発成果

○AI活用

- ・基本的には自分自身で作成
- ・コードの改善案や他の実装案の取得に利用

○成果

- ・C言語によるゲーム開発経験
- ・CUIによるゲーム開発経験
- ・キーボード入力に応じた処理

自己学習ポートフォリオ

研修期間中に自主学習で制作したゲーム作品

トランプゲーム(CUI)

氏名：N.K.
株式会社アルテクナ
アウトソーシング事業部

```
void sort_hand(Card hand[], int size){ //♠→♥→♣→♦の順に並び替え
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            // suit優先、次にnumber
            if(hand[i].suit > hand[j].suit || (hand[i].suit == hand[j].suit && hand[i].number > hand[j].number)){
                Card temp = hand[i];
                hand[i] = hand[j];
                hand[j] = temp;
            }
        }
    }
}
```

コード概要

○コードの総数

5966行

○コードの内訳

mainや変数定義：392行
ブラックジャック：570行
七並べ：379行
ババ抜き：446行
UNO：779行
大富豪：3400行

抜粋コードの概要

○目的

手札の表示順をスート順
(♠→♥→♣→♦)
に整列する (七並べ)

○引数

Card hand[]：手札の情報
int size：手札の枚数

○使用変数

suit：マーク(0：♠,1：♥,2：♣,3：♦)
number：数字(1~13)
temp：仮の代入カード

作品の今後に向けて

○実装出来なかった機能

- ・対人对戦機能
(理由：お互いの手札が見えてしまうため)
- ・大富豪以外でのローカルルール
例：七並べでのジョーカーの実装
(理由：実装すると元のゲームに悪影響を及ぼすことがあった)

○今後実装したい機能

- ・CPUレベルの設定
→レベルによってCPUの戦法を変える

自己学習ポートフォリオ

研修期間中に自主学習で制作したゲーム作品

トランプゲーム(GUI)

氏名：N.K.
株式会社アルテクナ
アウトソーシング事業部

概要

○開発内容

神経衰弱、ソリティア、
スパイダー

○開発目的

- ・GUIプログラミングの基礎を習得するため
- ・マウス操作を用いたプログラムを作るため

○開発期間

11日間

○開発人数

1名(個人開発)

○言語/UI

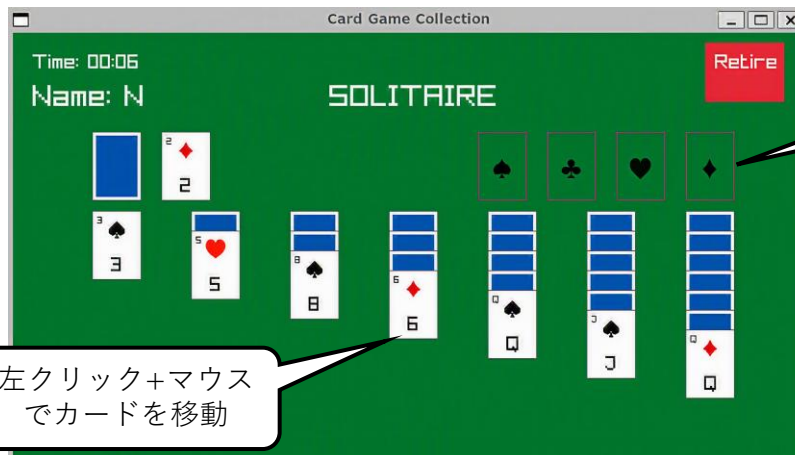
C言語/GUI

○ライブラリ

C標準ライブラリ、raylib

○開発環境

Visual Studio Code、
Linux(Ubuntu)



開発のポイント

○工夫した点

- ・左クリックによる選択処理と右クリックによる操作処理を実装
- ・ゲームのルールに基づいたカードの移動可能判定を実装

○苦労した点

- ・カード移動時の座標管理

問題点

- ・カード移動後の座標が更新されず元の位置に戻ってしまう
- ・移動不可の場所にカードを移動させたときに元の位置に戻らない

解決策

- 座標更新：移動可能なカードの場合、列の一番下の座標に更新させる
- 戻る処理：移動可否の判定をし、不可の場合は座標更新しない

開発成果

○AI活用

- ・基本的には自分自身で作成
- ・コードの改善案やデザイン案の取得に利用

○成果

- ・C言語によるゲーム開発経験
- ・GUIによるゲーム開発経験
- ・マウス、キーボード入力に応じた処理

自己学習ポートフォリオ

研修期間中に自主学習で制作したゲーム作品

氏名：N.K.
株式会社アルテクナ
アウトソーシング事業部

トランプゲーム(GUI)

```
if(IsMouseButtonPressed(MOUSE_RIGHT_BUTTON)){
    // waste → foundation
    if(wasteCount > 0 && CheckCollisionPointRec(mouse, waste[wasteCount-1].rect)){...

    // table → foundation
    for(int col=0; col<7; col++){
        int top = tableCount[col] - 1;
        if(top >= 0 && table[col][top].opened && CheckCollisionPointRec(mouse, table[col][top].rect)){
            Card card = table[col][top];

            if(solitaire_canMoveToFoundation(card)){
                int suit = card.suit;
                foundation[suit][foundationCount[suit]] = card;
                foundationCount[suit]++;
                tableCount[col]--;

                if(tableCount[col] > 0){
                    table[col][tableCount[col]-1].opened = 1;
                }
            }
        }
    }
}
```

```
bool solitaire_canMoveToFoundation(Card card){ //場置き判定
    int suit = card.suit;

    // Aなら空のfoundationへ
    if(foundationCount[suit] == 0){
        return card.number == 1;
    }

    // 次の数字ならOK
    Card top = foundation[suit][foundationCount[suit]-1];

    return card.number == top.number + 1;
}
```

コード概要

○コードの総数

3155行

○コードの内訳

Mainや変数定義：

484行

神経衰弱：

966行

ソリティア：

799行

スパイダー：

906行

抜粋コードの概要

○図の意味

左：右クリックで置き場にカードを移動する処理
右：置き場にカードを置くことが出来るかの判定

○目的

右クリックで置き場にカードを移動する
(ソリティア)

○引数/返り値

Card card：カードの情報
card.number：置き場のカードの数

○使用変数

suit：マーク(0：♠,1：♥,2：♣,3：♦)
number：数字(1~13)
foundation：カード置き場
table：縦に並んで重なっているカードの盤面

作品の今後に向けて

○実装出来なかった機能

- ・ ヒント機能
(理由：適度なヒントの設定と、ヒント使用によるペナルティの設定が困難なため)
- ・ マウスを分けた対戦機能
(理由：複数のマウス座標の判定が困難なため)

○今後実装したい機能

- ・ 1手戻す機能(スパイダー以外)
→ 盤面を毎回保存して
1手戻せるようにする